



Lab 5: Laplace filter

MCHA4400

Semester 2 2024

Introduction

In this lab, you will do the following:

1. Complete the implementation of C++ classes for working with Gaussian distributions in square-root moment form.
2. Implement the process dynamics and measurement model for a ballistic state estimation problem.
3. Run a square-root Laplace filter.
4. Plot the results using VTK.

The lab should be completed within 4 hours. The assessment will be done in the lab at the end of your enrolled lab session(s). Once you complete the tasks, call the lab demonstrator to start your assessment.

The lab is worth 5% of your course grade and is graded from 0–5 marks.



GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

1 Gaussian filtering (3 marks)

A Gaussian filter is an online state estimator that assumes a Gaussian state distribution, which is updated according to a stochastic differential equation describing the state dynamics (process model) and a sensor models (measurement likelihood functions).

If we parameterise the state Gaussian distribution by its first two moments—mean and covariance—we obtain a family of filters that includes the Laplace filter. With the additional assumption of a Gaussian measurement likelihood, we also obtain the nonlinear Kalman filters (e.g., EKF and UKF). Filters that propagate the moments of Gaussian distributions should be implemented in square-root form.

Let $\mathbf{x} \in \mathbb{R}^n$ and let

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{P}) = \frac{1}{\sqrt{\det 2\pi\mathbf{P}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{P}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right) \quad (1)$$

be a Gaussian distribution in moment form, where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\mathbf{P} \in \mathbb{R}^{n \times n}$ is the symmetric positive-definite covariance matrix. This Gaussian distribution can also be parameterised in square-root moment form as follows:

$$\mathcal{N}^{\frac{1}{2}}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{S}) = \frac{1}{|\det \sqrt{2\pi}\mathbf{S}|} \exp\left(-\frac{1}{2}\left\|\mathbf{S}^{-T}(\mathbf{x} - \boldsymbol{\mu})\right\|^2\right), \quad (2)$$

where $\mathbf{S} \in \mathbb{R}^{n \times n}$ is an upper-triangular matrix that satisfies the following relation with the covariance matrix:

$$\mathbf{S}^\top \mathbf{S} = \mathbf{P}. \quad (3)$$

In the Laplace filter, the k^{th} measurement correction seeks the state value that maximises the posterior density $p(\mathbf{x}_k | \mathbf{y}_{1:k})$, given the measurement likelihood $p(\mathbf{y}_k | \mathbf{x}_k)$ and the prior density $p(\mathbf{x}_k | \mathbf{y}_{1:k-1})$,

$$\begin{aligned} \boldsymbol{\mu}_{k|k} &= \arg \max_{\mathbf{x}_k} p(\mathbf{x}_k | \mathbf{y}_{1:k}) \\ &= \arg \max_{\mathbf{x}_k} \frac{p(\mathbf{y}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{y}_{1:k-1})}{p(\mathbf{y}_k | \mathbf{y}_{1:k-1})} \\ &= \arg \max_{\mathbf{x}_k} p(\mathbf{y}_k | \mathbf{x}_k) p(\mathbf{x}_k | \mathbf{y}_{1:k-1}) \\ &= \arg \max_{\mathbf{x}_k} p(\mathbf{y}_k | \mathbf{x}_k) \mathcal{N}(\mathbf{x}_k; \boldsymbol{\mu}_{k|k-1}, \mathbf{P}_{k|k-1}). \end{aligned}$$

In practice, it is more convenient to minimise the negative log of the above cost, i.e., let

$$\mathcal{V}(\mathbf{x}_k) = -\log p(\mathbf{y}_k | \mathbf{x}_k) - \log \mathcal{N}(\mathbf{x}_k; \boldsymbol{\mu}_{k|k-1}, \mathbf{P}_{k|k-1}). \quad (4)$$

Then, under the Laplace approximation, the posterior mean and covariance are given by

$$\begin{aligned} \boldsymbol{\mu}_{k|k} &= \arg \min_{\mathbf{x}_k} \mathcal{V}(\mathbf{x}_k), \\ \mathbf{P}_{k|k} &= \left(\frac{\partial^2 \mathcal{V}}{\partial \mathbf{x}_k^2} \bigg|_{\mathbf{x}_k = \boldsymbol{\mu}_{k|k}} \right)^{-1}. \end{aligned}$$

The optimisation to find $\boldsymbol{\mu}_{k|k}$ and $\mathbf{P}_{k|k}$ requires the gradient and Hessian,

$$\mathbf{g}_k = \frac{\partial \mathcal{V}}{\partial \mathbf{x}_k} = -\frac{\partial}{\partial \mathbf{x}_k} \log p(\mathbf{y}_k | \mathbf{x}_k) - \frac{\partial}{\partial \mathbf{x}_k} \log \mathcal{N}(\mathbf{x}_k; \boldsymbol{\mu}_{k|k-1}, \mathbf{P}_{k|k-1}), \quad (5)$$

$$\mathbf{H}_k = \frac{\partial^2 \mathcal{V}}{\partial \mathbf{x}_k^2} = -\frac{\partial^2}{\partial \mathbf{x}_k^2} \log p(\mathbf{y}_k | \mathbf{x}_k) - \frac{\partial^2}{\partial \mathbf{x}_k^2} \log \mathcal{N}(\mathbf{x}_k; \boldsymbol{\mu}_{k|k-1}, \mathbf{P}_{k|k-1}), \quad (6)$$

respectively.

Info

The cost (4), gradient (5) and Hessian (6) are implemented in `Measurement::costJointDensity`, which rely on a descendant class to implement `Measurement::logLikelihood`, which evaluates the log likelihood and its derivatives for a particular sensor type. The optimisation problem itself is solved in `Measurement::update`.

Since the prior is Gaussian, we therefore need a function to evaluate the log of a Gaussian distribution and its first two derivatives. In addition, if the measurement likelihood is also Gaussian, i.e.,

$$p(\mathbf{y}_k | \mathbf{x}_k) = \mathcal{N}(\mathbf{y}_k; \mathbf{h}(\mathbf{x}_k), \mathbf{R}),$$

for some mean function $\mathbf{h}(\cdot)$ and covariance $\mathbf{R}$, then we also need to be able to compute the derivatives, $\frac{\partial}{\partial \mathbf{x}_k} \log \mathcal{N}(\mathbf{y}_k; \mathbf{h}(\mathbf{x}_k), \mathbf{R})$ and $\frac{\partial^2}{\partial \mathbf{x}_k^2} \log \mathcal{N}(\mathbf{y}_k; \mathbf{h}(\mathbf{x}_k), \mathbf{R})$.

1.1 Log likelihood

The log of a Gaussian distribution in square-root moment form (2) is given by

$$\begin{aligned}
 \log \mathcal{N}^{\frac{1}{2}}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{S}) &= \log \left((\det 2\pi \mathbf{S}^T \mathbf{S})^{-\frac{1}{2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T (\mathbf{S}^T \mathbf{S})^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right) \right) \\
 &= -\frac{1}{2} \log(\det 2\pi \mathbf{S}^T \mathbf{S}) - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{S}^{-1} \mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}) \\
 &= -\frac{1}{2} \log((2\pi)^n (\det \mathbf{S})^2) - \frac{1}{2} (\mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}))^T \mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}) \\
 &= -\frac{n}{2} \log 2\pi - \log |\det \mathbf{S}| - \frac{1}{2} \left\| \mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}) \right\|^2 \\
 &= -\frac{n}{2} \log 2\pi - \log \left| \prod_{i=1}^n \mathbf{S}_{ii} \right| - \frac{1}{2} \|\mathbf{w}\|^2 \\
 &= -\frac{n}{2} \log 2\pi - \sum_{i=1}^n \log |\mathbf{S}_{ii}| - \frac{1}{2} \mathbf{w}^T \mathbf{w},
 \end{aligned} \tag{7}$$

where $|\cdot|_{ii}$ denotes the absolute value of the i^{th} diagonal element of its argument and $\mathbf{w}$ is the solution to the triangular system of equations $\mathbf{S}^T \mathbf{w} = \mathbf{x} - \boldsymbol{\mu}$.

Tasks

- Prepare the build directory for a debug build with documentation generation enabled and run the unit tests as follows:

Terminal

```

nerd@basement:~/MCHA4400/lab5$ cmake -G Ninja -B build -DBUILD_DOCUMENTATION=ON -DCMAKE_BUILD_TYPE=Debug
↳ && cd build
nerd@basement:~/MCHA4400/lab5/build$ ninja
[Compiling intensifies]
[doctest] test cases: 11 | 11 passed | 0 failed | 17 skipped
[doctest] assertions: 602 | 602 passed | 0 failed |
[doctest] Status: SUCCESS!
    
```



Tip

To configure the project in VS Code, the `BUILD_DOCUMENTATION` flag can be set in `.vscode/settings.json`. It is already set to `ON` for this project.

The project is now configured to automatically generate documentation from the source code using `doxygen` as part of the build process. This can be accessed by opening `lab5/docs/html/index.html` in your web browser, which may be helpful for navigating the codebase. The style should look familiar, as `doxygen` is also used to generate the `Eigen` and `OpenCV` documentation.

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/GaussianLog.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the naïve implementation of (7) provided.

- c) Complete a numerically robust implementation and its first two derivatives as the three `Gaussian::log` functions within `src/Gaussian.hpp`¹.



Tip

Since the members of `Gaussian` are dependent on the `Scalar` type, the compiler may need some extra hints by way of the `typename` and `template` keywords to help deduce if particular tokens are types or templates, respectively. For example, if you need an upper triangular view of `S_`, which is a dependent type `Eigen::MatrixX<Scalar>`, the `template` keyword may be required, i.e.,

```
S_.template triangularView<Eigen::Upper>().solve(/*...*/)
```

This is needed because type of `S_` is not known until the template is instantiated for a particular `Scalar` template argument, and prior to instantiation, the compiler cannot otherwise parse the syntax of the template.

- d) Run `ninja` and ensure the unit tests in `test/src/GaussianLog.cpp` pass.

1.2 Affine transform

Let $p(\mathbf{x}) = \mathcal{N}^{\frac{1}{2}}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{S})$ be a Gaussian distribution in square-root moment form, where $\mathbf{x}, \boldsymbol{\mu} \in \mathbb{R}^n$ and $\mathbf{S} \in \mathbb{R}^{n \times n}$ is an upper-triangular matrix. Then, let $\mathbf{h}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ be a map and define $\mathbf{y} = \mathbf{h}(\mathbf{x})$, where $\mathbf{y} \in \mathbb{R}^m$. Then, we can approximate $p(\mathbf{y})$ as

$$\begin{aligned} p(\mathbf{y}) &= \mathcal{N}(\mathbf{y}; \mathbf{h}(\boldsymbol{\mu}), \mathbf{C}\mathbf{S}^{\top}\mathbf{S}\mathbf{C}^{\top}) \\ &= \frac{1}{\sqrt{|2\pi\mathbf{C}\mathbf{S}^{\top}\mathbf{S}\mathbf{C}^{\top}|}} \exp\left(-\frac{1}{2}(\mathbf{y} - \mathbf{h}(\boldsymbol{\mu}))^{\top}(\mathbf{C}\mathbf{S}^{\top}\mathbf{S}\mathbf{C}^{\top})^{-1}(\mathbf{y} - \mathbf{h}(\boldsymbol{\mu}))\right), \end{aligned} \quad (8)$$

where $\mathbf{C} = \left. \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\boldsymbol{\mu}}$ is the Jacobian of $\mathbf{h}$ evaluated at $\boldsymbol{\mu}$.

The distribution (8) is almost in square-root moment form; however, $p(\mathbf{y}) \neq \mathcal{N}^{\frac{1}{2}}(\mathbf{y}; \mathbf{h}(\boldsymbol{\mu}), \mathbf{S}\mathbf{C}^{\top})$ since $\mathbf{S}\mathbf{C}^{\top}$ is not necessarily an upper-triangular matrix. To resolve this, let

$$\mathbf{S}\mathbf{C}^{\top} = \underbrace{\begin{bmatrix} \mathbf{Y} & \mathbf{Z} \end{bmatrix}}_{\mathbf{Q}} \underbrace{\begin{bmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{bmatrix}}_{\mathbf{R}}, \quad (9)$$

where $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix and $\mathbf{R} \in \mathbb{R}^{n \times m}$ is an upper-triangular matrix. Then,

$$\begin{aligned} (\mathbf{S}\mathbf{C}^{\top})^{\top}\mathbf{S}\mathbf{C}^{\top} &= \mathbf{R}^{\top}\mathbf{Q}^{\top}\mathbf{Q}\mathbf{R} = \mathbf{R}^{\top}\mathbf{R} && \text{(since } \mathbf{Q} \text{ is orthogonal)} \\ &= \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{bmatrix}^{\top} \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{bmatrix} \\ &= \mathbf{R}_1^{\top}\mathbf{R}_1. \end{aligned}$$

Therefore, the distribution of $\mathbf{y}$ in square-root moment form is given by

$$p(\mathbf{y}) = \mathcal{N}^{\frac{1}{2}}(\mathbf{y}; \mathbf{h}(\boldsymbol{\mu}), \mathbf{R}_1). \quad (10)$$

¹Template functions are implemented in a header, since they are recipes for building functions that are needed by all compilation units that include this header.

Tasks

- a) Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/GaussianTransform.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementation of (10) provided.
- b) Complete the implementation of the `Gaussian::affineTransform` template function in `Gaussian.hpp`
- c) Ensure that the unit tests provided in `test/src/GaussianTransform.cpp` pass.

1.3 Joint distribution

Let $\mathbf{x} \in \mathbb{R}^n$ and let

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{P}) \quad (11)$$

be a multivariate Gaussian distribution, where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\mathbf{P} \in \mathbb{R}^{n \times n}$ is the symmetric positive-definite covariance matrix. If we partition

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix}, \quad (12)$$

where the head $\mathbf{x}_a \in \mathbb{R}^{n_a}$ and the tail $\mathbf{x}_b \in \mathbb{R}^{n_b}$ with $n_a + n_b = n$, then we can write (11) as

$$p\left(\begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix}; \begin{bmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{bmatrix}, \begin{bmatrix} \mathbf{P}_{aa} & \mathbf{P}_{ab} \\ \mathbf{P}_{ba} & \mathbf{P}_{bb} \end{bmatrix}\right), \quad (13)$$

where $\boldsymbol{\mu}_a \in \mathbb{R}^{n_a}$, $\boldsymbol{\mu}_b \in \mathbb{R}^{n_b}$, $\mathbf{P}_{aa} \in \mathbb{R}^{n_a \times n_a}$, $\mathbf{P}_{ab} \in \mathbb{R}^{n_b \times n_a}$, $\mathbf{P}_{ba} \in \mathbb{R}^{n_b \times n_a}$ and $\mathbf{P}_{bb} \in \mathbb{R}^{n_b \times n_b}$.

Let us also parameterise (13) in square-root moment form as follows:

$$p\left(\begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix}\right) = \mathcal{N}^{\frac{1}{2}}\left(\begin{bmatrix} \mathbf{x}_a \\ \mathbf{x}_b \end{bmatrix}; \begin{bmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{bmatrix}, \begin{bmatrix} \mathbf{S}_1 & \mathbf{S}_2 \\ \mathbf{0} & \mathbf{S}_3 \end{bmatrix}\right), \quad (14)$$

where $\boldsymbol{\mu}_1 \in \mathbb{R}^{n_a}$, $\boldsymbol{\mu}_2 \in \mathbb{R}^{n_b}$, $\mathbf{S}_1 \in \mathbb{R}^{n_a \times n_a}$, $\mathbf{S}_2 \in \mathbb{R}^{n_a \times n_b}$, $\mathbf{S}_3 \in \mathbb{R}^{n_b \times n_b}$ and $\mathbf{S}_1$ and $\mathbf{S}_3$ are upper triangular.

Then, from (3), we have

$$\begin{aligned} \begin{bmatrix} \mathbf{P}_{aa} & \mathbf{P}_{ab} \\ \mathbf{P}_{ba} & \mathbf{P}_{bb} \end{bmatrix} &= \begin{bmatrix} \mathbf{S}_1 & \mathbf{S}_2 \\ \mathbf{0} & \mathbf{S}_3 \end{bmatrix}^T \begin{bmatrix} \mathbf{S}_1 & \mathbf{S}_2 \\ \mathbf{0} & \mathbf{S}_3 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{S}_1^T & \mathbf{0} \\ \mathbf{S}_2^T & \mathbf{S}_3^T \end{bmatrix} \begin{bmatrix} \mathbf{S}_1 & \mathbf{S}_2 \\ \mathbf{0} & \mathbf{S}_3 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{S}_1^T \mathbf{S}_1 & \mathbf{S}_1^T \mathbf{S}_2 \\ \mathbf{S}_2^T \mathbf{S}_1 & \mathbf{S}_2^T \mathbf{S}_2 + \mathbf{S}_3^T \mathbf{S}_3 \end{bmatrix}. \end{aligned} \quad (15)$$

We will use (15) to obtain results for the marginal and conditional distributions of a Gaussian distribution in square-root moment form in the following subsections.

1.3.1 Marginal distribution

The marginal distributions of the head $\mathbf{x}_a$ and the tail $\mathbf{x}_b$ are then given as follows:

$$\begin{aligned} p(\mathbf{x}_a) &= \mathcal{N}(\mathbf{x}_a; \boldsymbol{\mu}_a, \mathbf{P}_{aa}), \\ p(\mathbf{x}_b) &= \mathcal{N}(\mathbf{x}_b; \boldsymbol{\mu}_b, \mathbf{P}_{bb}). \end{aligned}$$

If we examine the marginal distribution of the head partition and compare the moment parameters to the square-root moment parameters, we find from (15) that

$$\mathbf{S}_1^\top \mathbf{S}_1 = \mathbf{P}_{aa}. \quad (16)$$

Therefore, the marginal distribution of the head partition of a Gaussian distribution in square-root moment form (14) is found simply from the partitions of its joint parameterisation as follows:

$$p(\mathbf{x}_a) = \mathcal{N}^{\frac{1}{2}}(\mathbf{x}_a; \boldsymbol{\mu}_a, \mathbf{S}_1). \quad (17)$$

If we want the marginal distribution of the tail partition, or any other subset of $\mathbf{x}$, we must permute that subset into the tail partition before employing (17).



Foreshadowing

In landmark SLAM, extracting the marginal distributions of a non-contiguous set of states is a common operation. At each time step, the marginal distribution of all landmarks that are predicted to be visible in the current frame is needed for correct data association with the measured image features in that frame. Since the order of the landmarks in the state is arbitrary, the states corresponding to visible landmarks do not form simple partitions of the state.

To do this, let us introduce an index vector $\mathcal{I} \in \mathbb{N}^{n_{\mathcal{I}}}$ to select the desired elements and denote this as $\mathbf{x}_{\mathcal{I}}$. Each element of $\mathbf{x}_{\mathcal{I}}$ is given by

$$(\mathbf{x}_{\mathcal{I}})_i = x_{\mathcal{I}_i}, \quad \text{for } i = 1, 2, \dots, n_{\mathcal{I}}.$$



Example

Let $\mathbf{x} = [x_1 \ x_2 \ x_3 \ x_4 \ x_5]^\top$ and let $\mathcal{I} = [3 \ 5 \ 1]$. Then $\mathbf{x}_{\mathcal{I}} = [x_3 \ x_5 \ x_1]^\top$.

The marginal distribution of $\mathbf{x}_{\mathcal{I}}$ in moment form is given by

$$p(\mathbf{x}_{\mathcal{I}}) = \mathcal{N}(\mathbf{x}_{\mathcal{I}}; \boldsymbol{\mu}_{\mathcal{I}}, \mathbf{P}_{\mathcal{I}\mathcal{I}}), \quad (18)$$

where $\boldsymbol{\mu}_{\mathcal{I}} \in \mathbb{R}^{n_{\mathcal{I}}}$ with elements given by $(\boldsymbol{\mu}_{\mathcal{I}})_i = \mu_{\mathcal{I}_i}$ for $i = 1, 2, \dots, n_{\mathcal{I}}$ and $\mathbf{P}_{\mathcal{I}\mathcal{I}} \in \mathbb{R}^{n_{\mathcal{I}} \times n_{\mathcal{I}}}$ with elements given by $(\mathbf{P}_{\mathcal{I}\mathcal{I}})_{i,j} = P_{\mathcal{I}_i, \mathcal{I}_j}$ for $i, j = 1, 2, \dots, n_{\mathcal{I}}$.

Example

Let

$$\boldsymbol{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \\ \mu_4 \\ \mu_5 \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} P_{11} & P_{12} & P_{13} & P_{14} & P_{15} \\ P_{21} & P_{22} & P_{23} & P_{24} & P_{25} \\ P_{31} & P_{32} & P_{33} & P_{34} & P_{35} \\ P_{41} & P_{42} & P_{43} & P_{44} & P_{45} \\ P_{51} & P_{52} & P_{53} & P_{54} & P_{55} \end{bmatrix},$$

and let $\mathcal{I} = [3 \ 5 \ 1]$. Then,

$$\boldsymbol{\mu}_{\mathcal{I}} = \begin{bmatrix} \mu_3 \\ \mu_5 \\ \mu_1 \end{bmatrix}, \quad \mathbf{P}_{\mathcal{I}\mathcal{I}} = \begin{bmatrix} P_{33} & P_{35} & P_{31} \\ P_{53} & P_{55} & P_{51} \\ P_{13} & P_{15} & P_{11} \end{bmatrix}.$$

Selecting $\mathcal{I}$ -rows and $\mathcal{I}$ -columns of $\mathbf{P}$ to form $\mathbf{P}_{\mathcal{I}\mathcal{I}}$ is equivalent to selecting *all*-rows and $\mathcal{I}$ -columns of $\mathbf{S}$, i.e.,

$$\mathbf{P}_{\mathcal{I}\mathcal{I}} = \mathbf{S}_{:, \mathcal{I}}^{\top} \mathbf{S}_{:, \mathcal{I}},$$

where $\mathbf{S}_{:, \mathcal{I}} \in \mathbb{R}^{n \times n_{\mathcal{I}}}$.

Since $\mathbf{S}_{:, \mathcal{I}}$ is a tall matrix that is not necessarily upper triangular, we can let

$$\mathbf{S}_{:, \mathcal{I}} = \underbrace{[\mathbf{Y} \ \mathbf{Z}]}_{\mathbf{Q}} \underbrace{\begin{bmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{bmatrix}}_{\mathbf{R}}$$

be a QR factorisation, where $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix and $\mathbf{R} \in \mathbb{R}^{n \times n_{\mathcal{I}}}$ is an upper triangular matrix. Then,

$$\begin{aligned} \mathbf{S}_{:, \mathcal{I}}^{\top} \mathbf{S}_{:, \mathcal{I}} &= \mathbf{R}^{\top} \mathbf{Q}^{\top} \mathbf{Q} \mathbf{R} = \mathbf{R}^{\top} \mathbf{R} && \text{(since } \mathbf{Q} \text{ is orthogonal)} \\ &= \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{bmatrix}^{\top} \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{bmatrix} \\ &= \mathbf{R}_1^{\top} \mathbf{R}_1. \end{aligned}$$

Therefore, the marginal distribution of $\mathbf{x}_{\mathcal{I}}$ in square-root moment form is given by

$$p(\mathbf{x}_{\mathcal{I}}) = \mathcal{N}^{\frac{1}{2}}(\mathbf{x}_{\mathcal{I}}; \boldsymbol{\mu}_{\mathcal{I}}, \mathbf{R}_1). \quad (19)$$

Info

Note that the closer $\mathcal{I}$ is to indexing a head partition of $\mathbf{x}$, the cheaper the QR decomposition becomes, becoming essentially free in the case where $\mathcal{I} = [1 \ 2 \ 3 \ \dots \ n_{\mathcal{I}}]$ exactly indexes a head partition, since $\mathbf{S}_{:, \mathcal{I}}$ is already upper triangular in this case.

Tasks

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/GaussianMarginal.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementation of (19) provided.
- Complete the implementation of the `Gaussian::marginal` template function in `Gaussian.hpp`.
- Ensure that the unit tests provided in `test/src/GaussianMarginal.cpp` pass.

1.3.1 Conditional distribution

Consider the joint distribution (13) partitioned into a head partition $\mathbf{x}_a$ and a tail partition $\mathbf{x}_b$. Then, the conditional distributions of the head given the tail, and the tail given the head, respectively, are given by

$$p(\mathbf{x}_a|\mathbf{x}_b) = \mathcal{N}(\mathbf{x}_a; \underbrace{\boldsymbol{\mu}_a + \mathbf{P}_{ab}\mathbf{P}_{bb}^{-1}(\mathbf{x}_b - \boldsymbol{\mu}_b)}_{\boldsymbol{\mu}_{a|b}}, \underbrace{\mathbf{P}_{aa} - \mathbf{P}_{ab}\mathbf{P}_{bb}^{-1}\mathbf{P}_{ba}}_{\mathbf{P}_{a|b}}), \quad (20a)$$

$$p(\mathbf{x}_b|\mathbf{x}_a) = \mathcal{N}(\mathbf{x}_b; \underbrace{\boldsymbol{\mu}_b + \mathbf{P}_{ba}\mathbf{P}_{aa}^{-1}(\mathbf{x}_a - \boldsymbol{\mu}_a)}_{\boldsymbol{\mu}_{b|a}}, \underbrace{\mathbf{P}_{bb} - \mathbf{P}_{ba}\mathbf{P}_{aa}^{-1}\mathbf{P}_{ab}}_{\mathbf{P}_{b|a}}). \quad (20b)$$

If we examine the conditional distribution $p(\mathbf{x}_b|\mathbf{x}_a)$ and compare the moment parameters to the square-root moment parameters, we find from (15) that

$$\begin{aligned} \mathbf{P}_{b|a} &= \mathbf{P}_{bb} - \mathbf{P}_{ba}\mathbf{P}_{aa}^{-1}\mathbf{P}_{ab} \\ &= \underbrace{\mathbf{S}_2^T\mathbf{S}_2 + \mathbf{S}_3^T\mathbf{S}_3}_{\mathbf{P}_{bb}} - \underbrace{\mathbf{S}_2^T\mathbf{S}_1}_{\mathbf{P}_{ba}} \underbrace{(\mathbf{S}_1^T\mathbf{S}_1)^{-1}}_{\mathbf{P}_{aa}^{-1}} \underbrace{\mathbf{S}_1^T\mathbf{S}_2}_{\mathbf{P}_{ab}} \\ &= \mathbf{S}_2^T\mathbf{S}_2 + \mathbf{S}_3^T\mathbf{S}_3 - \mathbf{S}_2^T\mathbf{S}_1\mathbf{S}_1^{-1}\mathbf{S}_1^{-T}\mathbf{S}_1^T\mathbf{S}_2 \\ &= \mathbf{S}_2^T\mathbf{S}_2 + \mathbf{S}_3^T\mathbf{S}_3 - \mathbf{S}_2^T\mathbf{S}_2 \\ &= \mathbf{S}_3^T\mathbf{S}_3, \end{aligned}$$

and

$$\begin{aligned} \boldsymbol{\mu}_{b|a} &= \boldsymbol{\mu}_b + \mathbf{P}_{ba}\mathbf{P}_{aa}^{-1}(\mathbf{x}_a - \boldsymbol{\mu}_a) \\ &= \boldsymbol{\mu}_b + \underbrace{\mathbf{S}_2^T\mathbf{S}_1}_{\mathbf{P}_{ba}} \underbrace{(\mathbf{S}_1^T\mathbf{S}_1)^{-1}}_{\mathbf{P}_{aa}^{-1}}(\mathbf{x}_a - \boldsymbol{\mu}_a) \\ &= \boldsymbol{\mu}_b + \mathbf{S}_2^T\mathbf{S}_1\mathbf{S}_1^{-1}\mathbf{S}_1^{-T}(\mathbf{x}_a - \boldsymbol{\mu}_a) \\ &= \boldsymbol{\mu}_b + \mathbf{S}_2^T\mathbf{S}_1^{-T}(\mathbf{x}_a - \boldsymbol{\mu}_a). \end{aligned}$$

Therefore, the conditional distribution $p(\mathbf{x}_a|\mathbf{x}_b)$ in square-root moment form is given as follows:

$$p(\mathbf{x}_b|\mathbf{x}_a) = \mathcal{N}^{\frac{1}{2}}(\mathbf{x}_b; \boldsymbol{\mu}_b + \mathbf{S}_2^T\mathbf{S}_1^{-T}(\mathbf{x}_a - \boldsymbol{\mu}_a), \mathbf{S}_3). \quad (21)$$

If we want the conditional distribution of the head partition, or any other subset of $\mathbf{x}$, we must permute that subset into the tail partition before employing (21). To do this, let us introduce the index vectors

$\mathcal{A} \in \mathbb{N}^{n_{\mathcal{A}}}$ and $\mathcal{B} \in \mathbb{N}^{n_{\mathcal{B}}}$ to select the desired elements of each set and denote these as $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$, respectively.

The conditional distribution of $\mathbf{x}_{\mathcal{A}}$ given $\mathbf{x}_{\mathcal{B}}$ is given by

$$p(\mathbf{x}_{\mathcal{A}}|\mathbf{x}_{\mathcal{B}}) = \mathcal{N}(\mathbf{x}_{\mathcal{A}}; \boldsymbol{\mu}_{\mathcal{A}|\mathcal{B}}, \mathbf{P}_{\mathcal{A}|\mathcal{B}}),$$

where

$$\begin{aligned}\boldsymbol{\mu}_{\mathcal{A}|\mathcal{B}} &= \boldsymbol{\mu}_{\mathcal{A}} + \mathbf{P}_{\mathcal{A}\mathcal{B}}\mathbf{P}_{\mathcal{B}\mathcal{B}}^{-1}(\mathbf{x}_{\mathcal{B}} - \boldsymbol{\mu}_{\mathcal{B}}), \\ \mathbf{P}_{\mathcal{A}|\mathcal{B}} &= \mathbf{P}_{\mathcal{A}\mathcal{A}} - \mathbf{P}_{\mathcal{A}\mathcal{B}}\mathbf{P}_{\mathcal{B}\mathcal{B}}^{-1}\mathbf{P}_{\mathcal{B}\mathcal{A}}.\end{aligned}$$

To obtain this conditional distribution in square-root form, we permute the elements as follows:

$$p\left(\begin{bmatrix} \mathbf{x}_{\mathcal{B}} \\ \mathbf{x}_{\mathcal{A}} \end{bmatrix}\right) = \mathcal{N}\left(\begin{bmatrix} \mathbf{x}_{\mathcal{B}} \\ \mathbf{x}_{\mathcal{A}} \end{bmatrix}; \begin{bmatrix} \boldsymbol{\mu}_{\mathcal{B}} \\ \boldsymbol{\mu}_{\mathcal{A}} \end{bmatrix}, \begin{bmatrix} \mathbf{P}_{\mathcal{B}\mathcal{B}} & \mathbf{P}_{\mathcal{B}\mathcal{A}} \\ \mathbf{P}_{\mathcal{A}\mathcal{B}} & \mathbf{P}_{\mathcal{A}\mathcal{A}} \end{bmatrix}\right). \quad (22)$$

Selecting $\mathcal{B}$ and $\mathcal{A}$ rows and columns of $\mathbf{P}$ is equivalent to selecting the $\mathcal{B}$ and $\mathcal{A}$ *columns* of $\mathbf{S}$, i.e.,

$$\begin{bmatrix} \mathbf{P}_{\mathcal{B}\mathcal{B}} & \mathbf{P}_{\mathcal{B}\mathcal{A}} \\ \mathbf{P}_{\mathcal{A}\mathcal{B}} & \mathbf{P}_{\mathcal{A}\mathcal{A}} \end{bmatrix} = [\mathbf{S}_{:, \mathcal{B}} \quad \mathbf{S}_{:, \mathcal{A}}]^{\top} [\mathbf{S}_{:, \mathcal{B}} \quad \mathbf{S}_{:, \mathcal{A}}],$$

where $\mathbf{S}_{:, \mathcal{A}} \in \mathbb{R}^{n \times n_{\mathcal{A}}}$ and $\mathbf{S}_{:, \mathcal{B}} \in \mathbb{R}^{n \times n_{\mathcal{B}}}$.

Since $[\mathbf{S}_{:, \mathcal{B}} \quad \mathbf{S}_{:, \mathcal{A}}]$ is not necessarily upper triangular, we can let

$$[\mathbf{S}_{:, \mathcal{B}} \quad \mathbf{S}_{:, \mathcal{A}}] = \mathbf{Q} \underbrace{\begin{bmatrix} \mathbf{R}_1 & \mathbf{R}_2 \\ \mathbf{0} & \mathbf{R}_3 \end{bmatrix}}_{\mathbf{R}}$$

be a QR factorisation, where $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix and $\mathbf{R} \in \mathbb{R}^{n \times n}$ is an upper triangular matrix with blocks $\mathbf{R}_1 \in \mathbb{R}^{n_{\mathcal{B}} \times n_{\mathcal{B}}}$, $\mathbf{R}_2 \in \mathbb{R}^{n_{\mathcal{B}} \times n_{\mathcal{A}}}$ and $\mathbf{R}_3 \in \mathbb{R}^{n_{\mathcal{A}} \times n_{\mathcal{A}}}$, where $\mathbf{R}_1$ and $\mathbf{R}_3$ are upper triangular. Then,

$$\begin{aligned}[\mathbf{S}_{:, \mathcal{B}} \quad \mathbf{S}_{:, \mathcal{A}}]^{\top} [\mathbf{S}_{:, \mathcal{B}} \quad \mathbf{S}_{:, \mathcal{A}}] &= \mathbf{R}^{\top} \mathbf{Q}^{\top} \mathbf{Q} \mathbf{R} = \mathbf{R}^{\top} \mathbf{R} && \text{(since } \mathbf{Q} \text{ is orthogonal)} \\ &= \begin{bmatrix} \mathbf{R}_1 & \mathbf{R}_2 \\ \mathbf{0} & \mathbf{R}_3 \end{bmatrix}^{\top} \begin{bmatrix} \mathbf{R}_1 & \mathbf{R}_2 \\ \mathbf{0} & \mathbf{R}_3 \end{bmatrix}.\end{aligned}$$

$$p\left(\begin{bmatrix} \mathbf{x}_{\mathcal{B}} \\ \mathbf{x}_{\mathcal{A}} \end{bmatrix}\right) = \mathcal{N}^{\frac{1}{2}}\left(\begin{bmatrix} \mathbf{x}_{\mathcal{B}} \\ \mathbf{x}_{\mathcal{A}} \end{bmatrix}; \begin{bmatrix} \boldsymbol{\mu}_{\mathcal{B}} \\ \boldsymbol{\mu}_{\mathcal{A}} \end{bmatrix}, \begin{bmatrix} \mathbf{R}_1 & \mathbf{R}_2 \\ \mathbf{0} & \mathbf{R}_3 \end{bmatrix}\right). \quad (23)$$

Finally, the conditional distribution of $\mathbf{x}_{\mathcal{A}}$ given $\mathbf{x}_{\mathcal{B}}$ is given by the following:

$$p(\mathbf{x}_{\mathcal{A}}|\mathbf{x}_{\mathcal{B}}) = \mathcal{N}^{\frac{1}{2}}(\mathbf{x}_{\mathcal{A}}; \underbrace{\boldsymbol{\mu}_{\mathcal{A}} + \mathbf{R}_2^{\top} \mathbf{R}_1^{-\top} (\mathbf{x}_{\mathcal{B}} - \boldsymbol{\mu}_{\mathcal{B}})}_{\boldsymbol{\mu}_{\mathcal{A}|\mathcal{B}}}, \underbrace{\mathbf{R}_3}_{\mathbf{S}_{\mathcal{A}|\mathcal{B}}}). \quad (24)$$

Info

Note that the closer $\mathcal{A}$ is to indexing a tail partition of $\mathbf{x}$ and the closer $\mathcal{B}$ is to indexing a head partition, the cheaper the QR decomposition becomes, becoming essentially free in the case where $\mathcal{A}$ and $\mathcal{B}$ exactly index tail and head partitions, respectively.

Tasks

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/GaussianConditional.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the faulty implementation of (24) provided.
- Complete the implementation of the `Gaussian::conditional` template function in `Gaussian.hpp`.
- Ensure that the unit tests provided in `test/src/GaussianConditional.cpp` pass.

1.4 Confidence region

The Bayesian credible region (BCR) or *confidence region* of a pdf $p(\cdot)$ is given by the set

$$\mathcal{R} = \{\mathbf{x} \in \mathcal{X} \mid p(\mathbf{x}) \geq t\}, \quad (25a)$$

where the density threshold, t , is the unique solution to

$$\int_{p(\mathbf{x}) \geq t} p(\mathbf{x}) \, d\mathbf{x} = c, \quad (25b)$$

where $0 < c < 1$ is the desired probability mass to be contained within the BCR. Common choices include $c = 0.95$ or $c = 0.997$, which correspond to the $\mu \pm 2\sigma$ and $\mu \pm 3\sigma$ regions of a Gaussian distribution, respectively.

If $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{P})$ where $\mathbf{x} \in \mathbb{R}^n$, then the BCR (25) is an n -dimensional hyperellipsoid that defined by the points $\mathbf{x}$ that satisfy the following inequality:

$$(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{P}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \leq \chi_n^2(c), \quad (26)$$

where $\chi_n^2(c)$ is the inverse cumulative distribution function for probability c of the chi-squared distribution with n degrees of freedom².

The points $\mathbf{x}$ that lie on the boundary of (26) satisfy the following equation:

$$\underbrace{\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}}_{\mathbf{p}^\top} \underbrace{\begin{bmatrix} \mathbf{P}^{-1} & -\mathbf{P}^{-1}\boldsymbol{\mu} \\ -\boldsymbol{\mu}^\top \mathbf{P}^{-1} & \boldsymbol{\mu}^\top \mathbf{P}^{-1} \boldsymbol{\mu} - \chi_n^2(c) \end{bmatrix}}_{\mathbf{Q}} \underbrace{\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}}_{\mathbf{p}} = 0, \quad (27)$$

which admits the following compact form in homogeneous coordinates:

$$\mathbf{p}^\top \mathbf{Q} \mathbf{p} = 0. \quad (28)$$

²In MATLAB, this is equivalent to $\chi_n^2(c) = \text{chi2inv}(c, n)$.

If we parameterise the covariance with an upper triangular matrix $\mathbf{S}$ such that $\mathbf{S}^T \mathbf{S} = \mathbf{P}$, then (26) can be written as

$$\begin{aligned} (\mathbf{x} - \boldsymbol{\mu})^T (\mathbf{S}^T \mathbf{S})^{-1} (\mathbf{x} - \boldsymbol{\mu}) &\leq \chi_n^2(c) \\ (\mathbf{x} - \boldsymbol{\mu})^T \mathbf{S}^{-1} \mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}) &\leq \chi_n^2(c) \\ \left(\mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}) \right)^T \mathbf{S}^{-T} (\mathbf{x} - \boldsymbol{\mu}) &\leq \chi_n^2(c) \\ \mathbf{w}^T \mathbf{w} &\leq \chi_n^2(c), \end{aligned} \tag{29}$$

where $\mathbf{w}$ is the solution to the following triangular system of equations:

$$\mathbf{S}^T \mathbf{w} = \mathbf{x} - \boldsymbol{\mu}. \tag{30}$$

The matrix that appears in the homogeneous equation (28) is given by the following:

$$\mathbf{Q} = \begin{bmatrix} \mathbf{S}^{-1} \mathbf{S}^{-T} & -\mathbf{y} \\ -\mathbf{y}^T & \mathbf{z}^T \mathbf{z} - \chi_n^2(c) \end{bmatrix}, \tag{31}$$

where $\mathbf{z}$ and $\mathbf{y}$ are the solutions to the following triangular systems of equations:

$$\begin{aligned} \mathbf{S}^T \mathbf{z} &= \boldsymbol{\mu}, \\ \mathbf{S} \mathbf{y} &= \mathbf{z}. \end{aligned}$$

Tasks

- a) Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/GaussianConfidence.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementations provided.
- b) Complete the implementation of $\chi_\nu^2(p)$ in the `GaussianBase::chi2inv` static function in `GaussianBase.hpp`, which is the inverse cumulative distribution function for probability p of the chi-squared distribution with ν degrees of freedom. To do this, you can use `boost::math::gamma_p_inv`, noting that

$$\text{chi2inv}(p, \nu) \equiv 2 \cdot \text{gamma_p_inv} \left(\frac{\nu}{2}, p \right). \tag{32}$$

Include the appropriate `boost` header file(s) within `src/GaussianBase.hpp` as needed.

- c) Complete the implementation of the `GaussianBase::normcdf` static function in `GaussianBase.hpp`, which evaluates the cumulative density function for a standard normal distribution³. To do this, you can use `boost::math::erfc`, noting that

$$\text{normcdf}(w) \equiv \frac{1}{2} \cdot \text{erfc} \left(\frac{-w}{\sqrt{2}} \right). \tag{33}$$

Include the appropriate `boost` header file(s) within `src/GaussianBase.hpp` as needed.

- d) Implement (29) in the `Gaussian::isWithinConfidenceRegion` member function in `Gaussian.hpp`.

³zero mean and unit variance

**Hint**

The probability mass, c , contained within n_σ standard deviations of the mean of a Gaussian pdf is given by

$$c = \int_{\mu - n_\sigma \sigma}^{\mu + n_\sigma \sigma} \mathcal{N}(x; \mu, \sigma^2) dx = \int_{-n_\sigma}^{n_\sigma} \mathcal{N}(x; 0, 1) dx, \quad (34)$$

which can be integrated numerically or computed using the normal cumulative distribution function.

- e) Complete the implementation of the `GaussianBase::confidenceEllipse` member function in `GaussianBase.hpp`, which returns a set of points on the elliptical boundary of a given confidence region for a bivariate ($n = 2$) Gaussian distribution.

**Hint**

For $n = 2$, $\mathbf{w} \in \mathbb{R}^2$ and the boundary of (29) becomes the equation of a circle with radius $\sqrt{\chi_n^2(c)}$. Points on this circle can be found in $\mathbf{w}$ -coordinates and then transformed back to $\mathbf{x}$ -coordinates using (30) to obtain points on the boundary of the confidence ellipse.

- f) Complete the implementation of (31) in the `Gaussian::quadricSurface` member function in `Gaussian.hpp`, which calculates the homogeneous matrix representation for the quadric surface $\mathbf{Q}$, for a trivariate ($n = 3$) Gaussian distribution.
- g) Build the application and ensure the unit tests in `test/src/GaussianConfidence.cpp` pass.

2 Ballistic state estimation (2 marks)

A SpaceY Starship is re-entering earth's atmosphere and a Gaussian filter is needed to estimate its motion. SpaceY engineers were not expecting the Starship to fall in the orientation depicted in Figure 1, as such, the drag coefficient needs to be estimated online along with the altitude and velocity of the rocket.

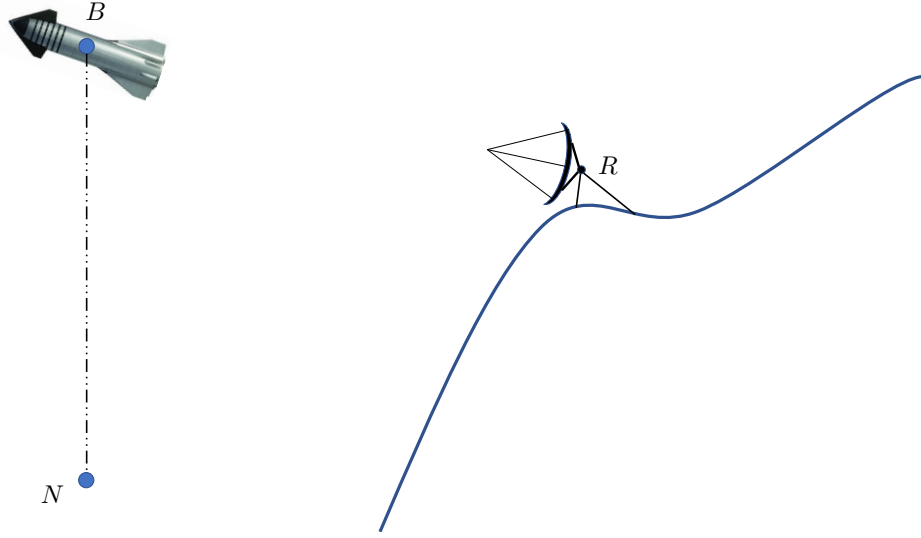


Figure 1: Vertical re-entry configuration of the SpaceY Starship

2.1 Ballistic process model

The following state vector has been proposed:

$$\mathbf{x} = \begin{bmatrix} h \\ v \\ c \end{bmatrix}, \quad (35)$$

where h is the altitude of the rocket above the reference point N , v is the velocity of the rocket and c is the ballistic drag coefficient to be estimated online.

The mean dynamics of the state vector can be expressed in the form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}), \quad (36)$$

where the dynamics of the free-falling Starship are given by the following expression:

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ f_3(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} v \\ d - g \\ 0 \end{bmatrix}. \quad (37)$$

The acceleration d due to drag is given by

$$d = \frac{1}{2} \rho v^2 c \quad (38)$$

and the air density is given by

$$\rho = \frac{pM}{RT}, \quad (39)$$

where M is the molar mass of air, R is the ideal gas constant and T is the air temperature.

If the air is in hydrostatic equilibrium, then an infinitesimal change in pressure dp can be related to a infinitesimal change in altitude dh as follows:

$$dp = -\rho g dh. \quad (40)$$

Dividing both sides by p and substituting (39) yields

$$\frac{dp}{p} = -\frac{\rho g}{p} dh = -\frac{gM}{RT} dh. \quad (41)$$

In the lower atmosphere, the temperature can be assumed to vary with altitude according to

$$T = T_0 - Lh, \quad (42)$$

where T_0 is the standard temperature and L is the temperature lapse rate. Substituting (42) into (41) and integrating yields

$$\begin{aligned} \frac{dp}{p} &= -\frac{gM}{R(T_0 - Lh)} dh \\ \int_{p_0}^p \frac{1}{p'} dp' &= -\frac{gM}{R} \int_0^h \frac{1}{T_0 - Lh'} dh' \\ [\log p']_{p'=p_0}^{p'=p} &= -\frac{gM}{R} \left[-\frac{1}{L} \log(T_0 - Lh') \right]_{h'=h_0}^{h'=h} \\ \log p - \log p_0 &= \frac{gM}{RL} (\log(T_0 - Lh) - \log T_0) \\ p &= p_0 \exp \left(\frac{gM}{RL} \log \left(1 - \frac{Lh}{T_0} \right) \right) \\ p &= p_0 \left(1 - \frac{Lh}{T_0} \right)^{\frac{gM}{RL}}, \end{aligned} \quad (43)$$

which is known as the barometric formula.

Substituting (43), (42) and (39) into (38) yields the drag acceleration as a function of states,

$$\begin{aligned} d &= \frac{1}{2} \rho v^2 c \\ d &= \frac{1}{2} \frac{pM}{RT} v^2 c \\ d &= \frac{1}{2} \frac{Mp_0}{R} \left(\frac{1}{T_0 - Lh} \right) \left(1 - \frac{Lh}{T_0} \right)^{\frac{gM}{RL}} v^2 c. \end{aligned} \quad (44)$$

where $p_0 = 101\,325$ Pa is the standard pressure at sea level, $T_0 = 288.15$ K is the standard temperature at sea level, $g = 9.81$ m/s² is the acceleration due to gravity, $M = 0.028\,964\,4$ kg/mol is the molar mass of dry air, $R = 8.314\,47$ J/(mol K) is the ideal gas constant and $L = 0.0065$ K/m is the temperature lapse rate in the lower atmosphere.

The process model for the rocket dynamics is then given by

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t)) + \dot{\mathbf{w}}(t), \quad \dot{\mathbf{w}}(t) \sim \mathcal{GP}(\mathbf{0}, \mathbf{Q} \delta(t - t')), \quad (45)$$

where the power spectral density of the process noise is

$$\mathbf{Q} = \text{diag} \left(\begin{bmatrix} 0 & 1 \times 10^{-20} & 25 \times 10^{-12} \end{bmatrix} \right). \quad (46)$$

Tasks

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/SystemBallistic.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementation provided.
- Complete the implementation of the three `SystemBallistic::dynamics` functions in `src/SystemBallistic.cpp`.
- Complete the implementations of the `SystemBallistic::processNoiseDensity` and `SystemBallistic::processNoiseIndex` functions in `src/SystemBallistic.cpp`
- Ensure that the unit tests provided in `test/src/SystemBallistic.cpp` pass.

2.2 Range measurement model (1 mark)

A range-only RADAR (ROR) unit provides range measurements of the rocket from position $(r_1, r_2) = (5000 \text{ m}, 5000 \text{ m})$. Therefore, the measurement model is given by

$$h_{\text{rng}}(\mathbf{x}) = \sqrt{r_1^2 + (h - r_2)^2}, \quad (47)$$

with Gaussian measurement noise of $\sigma_{\text{rng}} = 50 \text{ m}$.

Tasks

- Remove the `doctest::skip` decorator from each `SCENARIO` in `test/src/MeasurementRADAR.cpp` to enable these unit tests and run `ninja` to verify that they *fail* due to the incomplete implementation provided.
- Complete the implementation of the `MeasurementRADAR::predict` functions in `src/MeasurementRADAR.cpp`.
- Complete the implementation of `MeasurementRADAR::noise` in `src/MeasurementRADAR.cpp`.
- Ensure that the unit tests provided in `test/src/MeasurementRADAR.cpp` pass.

2.3 State estimation and visualisation

The framework for an event-based square-root Gaussian filter has been implemented in the `DensityBase`, `GaussianBase`, `Gaussian`, `SystemBase`, `SystemEstimator` `Event` and `Measurement` classes (see the `.h/.cpp` or `.hpp` files for each). The `System*` and `Measurement*` classes are extended to `SystemBallistic` and `MeasurementRADAR`, respectively, to provide the problem-specific implementations of the process model dynamics and measurement model. More formally, the `SystemBallistic` class inherits from the

`SystemEstimator` class, while the `MeasurementRADAR` class inherits from the `MeasurementGaussianLikelihood` class, which in turn inherits from the `Measurement` class, which in turn inherits from `Event` class.

The main loop for each time step of the state estimator is implemented in `src/main.cpp`, which creates each measurement event and then processes it. Measurement events are processed by performing a time update on the state to the time stamp of the measurement followed by a measurement update. This can be seen in the `Event::process`, `SystemBallistic::predict` and `Measurement::update` functions.

The results are plotted in the `plot_simulation` function in `src/ballistic_plot.cpp` using Visualisation Toolkit (VTK), which is a powerful 2D and 3D visualisation library.

Tasks

- a) Run the `lab5` application and ensure that you get a similar output to the following:

```
Terminal
nerd@basement:~/MCHA4400/lab5/build$ ninja && ./lab5
[Ninja-ing intensifies]
Reading data from ../data/estimationdata.csv
Found 501 rows within ../data/estimationdata.csv

Initial state estimate
mu[0] =
  14000
  -450
  0.0005
P[0] =
  4.84e+06      0      0
      0    10000      0
      0      0    1e-06
[t=000.000s] RADAR measurement update:..... done
[t=000.100s] RADAR measurement update:..... done
[t=000.200s] RADAR measurement update:..... done
[Filtering intensifies]
[t=049.800s] RADAR measurement update:.... done
[t=049.900s] RADAR measurement update:.... done
[t=050.000s] RADAR measurement update:.... done

Final state estimate
mu[end] =
  2185.02
  -152.732
  0.000982088
P[end] =
  216.724    10.7621  0.000163735
   10.7621    1.57176  2.55059e-05
  0.000163735  2.55059e-05  4.79729e-10
[A VTK plot window will display estimation results with some missing subplots]
```

- b) Complete the implementation of the `plot_simulation` function, to produce a figure similar to Figure 2. The plot should include:
- the true value of each state,
 - the estimated mean for each state,
 - the estimated 99.7% confidence region for each state,

- iv) The marginal standard deviations of each state on a separate subfigures with a vertical log axis.



Tip

The [VTK C++ examples](#) are a useful resource for getting started with VTK. Some of the following API documentation may also be useful:

- [vtkChartXY](#), for drawing 2-dimensional charts
- [vtkPlot](#), for plotting lines within a chart
- [vtkPlotArea](#), for plotting areas between two lines
- [vtkChartLegend](#), for modifying legends of chart objects
- [setLabel](#), for modifying labels of [vtkPlot](#) and [vtkPlotArea](#) objects.

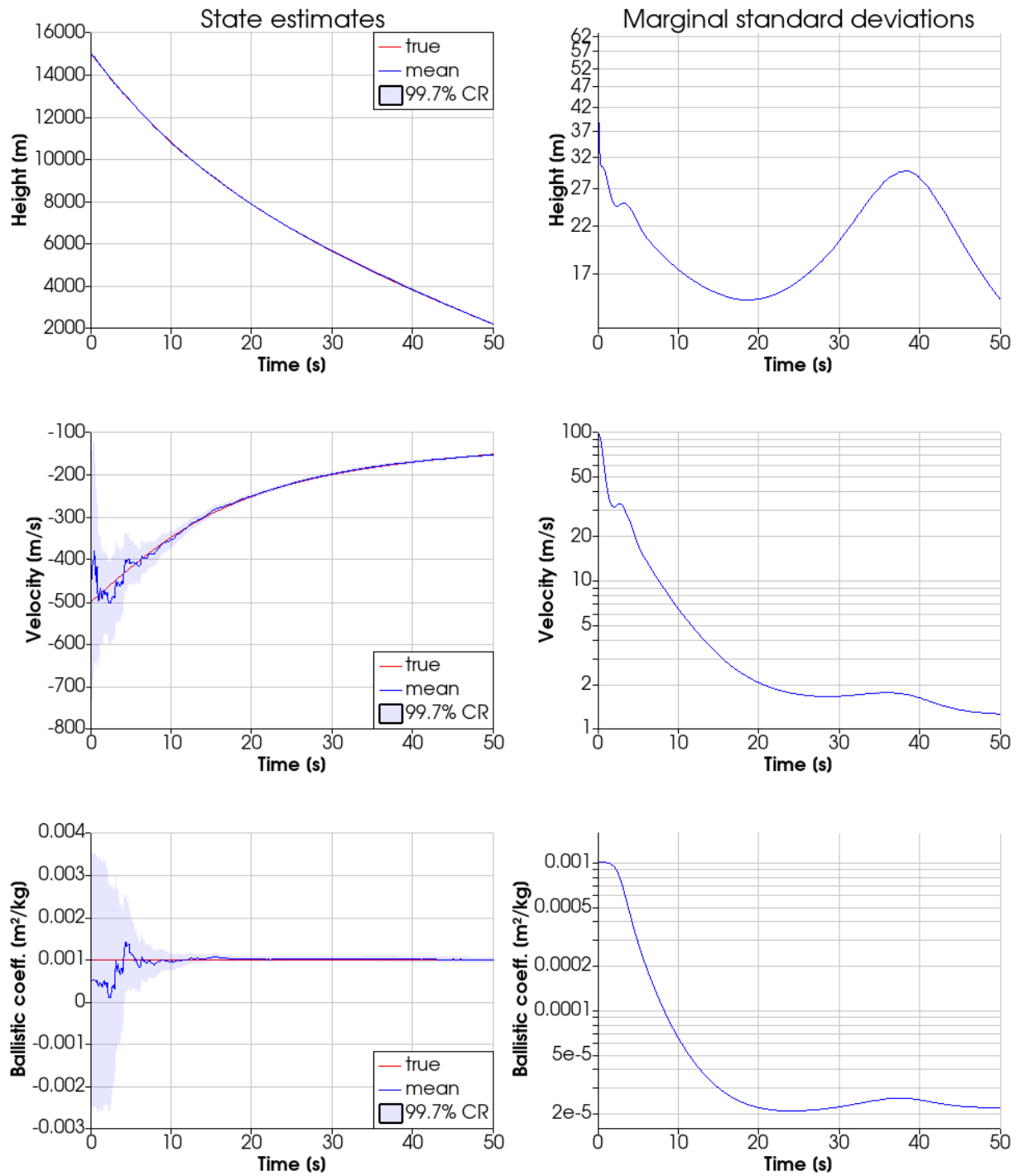


Figure 2: Expected plot.